Heat Flux Analysis

Time-series analysis of the variable HeatFlux_st extracted from 800 runs of GX.

In [1]:
```
import src.pugs as pugs

import numpy as np
import pandas as pd
import scipy.stats as st
import matplotlib.pyplot as plt

import os
from glob import glob
```

In [2]:
```
def process_folder(foler, exp_id, trim_variable, variable=None, output_file=
    # Get all experiment folders matching exp_id
    exp_folders = os.path.join(foler, f"{exp_id}_theta0_*_zeta0_*")
    folders = sorted(glob(exp_folders))

    # Process each folder and collect results
    results = []
    for f in folders:
        # Process individual GX file
        file = os.path.join(f, "gx.out.nc")
        output = pugs.process_file(trim_variable, file, variable=variable, m

        # Extract experiment identification
        parts = os.path.basename(f).split("_")
        exp_info = {"id": parts[0], "theta": parts[2], "zeta": parts[4]}

        # Skip if output is None
        if output is None:
            results.append({**exp_info, "file": file})
            continue

        # Read output CSV and extract results
        df = pd.read_csv(output)
        cols = df.columns
        for row in df.itertuples(index=False):
            results.append({**exp_info, **dict(zip(cols, row))})

        # Remove temporary output file
        os.remove(output)

    # Prepare output CSV
    if output_file is None:
        output_file = f"{exp_id}_{trim_variable}_{method}.csv"
        if exp_id == "*":
            output_file = f"all_{trim_variable}_{method}.csv"

    # Return results as DataFrame and save to CSV
    df = pd.DataFrame(results)
```

```
    output = pugs.export_file(df, output_file, format="csv")

    return output
```

In [3]:
```
folder = "/Users/zacharygottlieb/local/stellarator/data/20241119-01-008_200_
exp_id = "000089"
variable = "HeatFlux_st"
method = "linreg"
```

In [4]:
```
output = process_folder(folder, exp_id, trim_variable=variable, variable=var
df = pd.read_csv(output)
df.head()
```

Out[4]:

	id	theta	zeta	file	trim_variable	variable	method	start_time	
0	89	0.00	0.00	/Users/zacharygottlieb/local/stellarator/data/...	HeatFlux_st	HeatFlux_st	linreg	30.984904	27
1	89	0.00	1.57	/Users/zacharygottlieb/local/stellarator/data/...	HeatFlux_st	HeatFlux_st	linreg	40.158300	6.
2	89	3.14	0.00	/Users/zacharygottlieb/local/stellarator/data/...	HeatFlux_st	HeatFlux_st	linreg	35.718684	8.
3	89	3.14	1.57	/Users/zacharygottlieb/local/stellarator/data/...	HeatFlux_st	HeatFlux_st	linreg	40.218501	1.

4 rows × 21 columns

In [5]:
```
fig, axes = plt.subplots(2, 2, figsize=(16,6))

for i, ax in enumerate(axes.flatten()):
    df_i = pugs.load_file(df.iloc[i]["file"], variable)
    t = df_i["time"].to_numpy()
    x = df_i[variable].to_numpy()

    ax.plot(t, x)
    ax.axvline(df.iloc[i]["start_time"], color='red', linestyle='--', label=

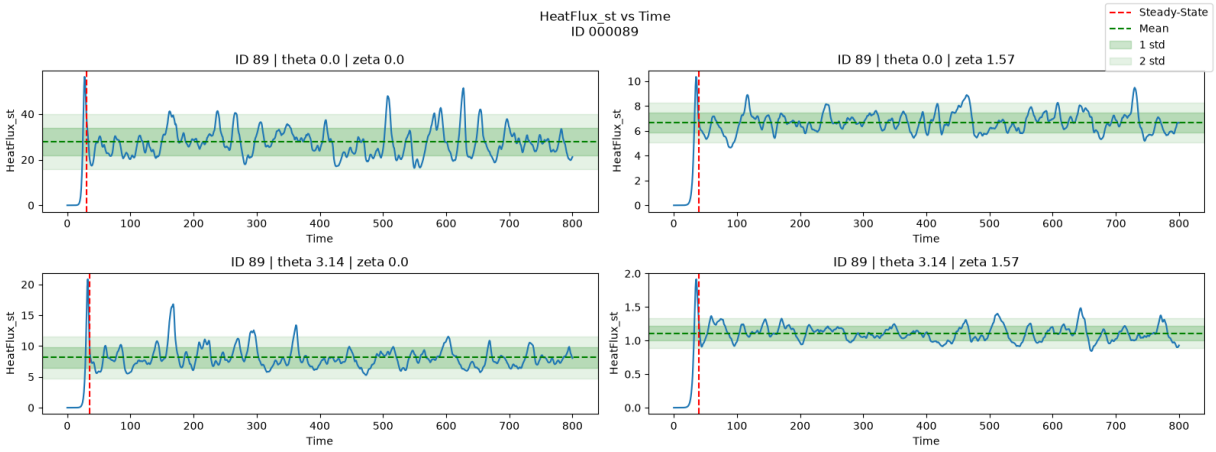
    ax.axhline(df.iloc[i]["mean"], color='green', linestyle='--', label="Mea
    ax.axhspan(df.iloc[i]["mean"] - df.iloc[i]["std"], df.iloc[i]["mean"] +
    ax.axhspan(df.iloc[i]["mean"] - 2 * df.iloc[i]["std"], df.iloc[i]["mean"

    ax.set_title(f"ID {df.iloc[i]['id']} | theta {df.iloc[i]['theta']} | zet
    ax.set_xlabel("Time")
    ax.set_ylabel(variable)

fig.suptitle(f"{variable} vs Time\nID {exp_id}")
```

```
handles, labels = axes.flatten()[-1].get_legend_handles_labels()
fig.legend(handles, labels, loc='upper right')

plt.tight_layout()
plt.show()
```



In [ ]:
```
process_folder(folder, "*", trim_variable=variable, variable=variable, metho
```

In [6]:
```
df_raw = pd.read_csv(f"all_{variable}_{method}.csv")
df = df_raw[df_raw["start_time"].notnull()]
print(f"{df.shape[0]} runs with steady-state found")

df_failed = df_raw[df_raw["start_time"].isnull()]
failed_init = []
failed_algo = []

for row in df_failed.itertuples(index=False):
    df_i = pugs.load_file(row.file, variable)
    x = df_i[variable].to_numpy()
    if np.max(x) < 1e-4:
        failed_init.append(row)
    else:
        failed_algo.append(row)

df_failed_init = pd.DataFrame(failed_init)
df_failed_algo = pd.DataFrame(failed_algo)

print(f"{df_failed_init.shape[0]} runs failed to initialize")
print(f"{df_failed_algo.shape[0]} runs failed to reach steady-state")
```

```
517 runs with steady-state found
269 runs failed to initialize
14 runs failed to reach steady-state
```

In [7]:
```
fig, axes = plt.subplots(3, 3, figsize=(16,8))

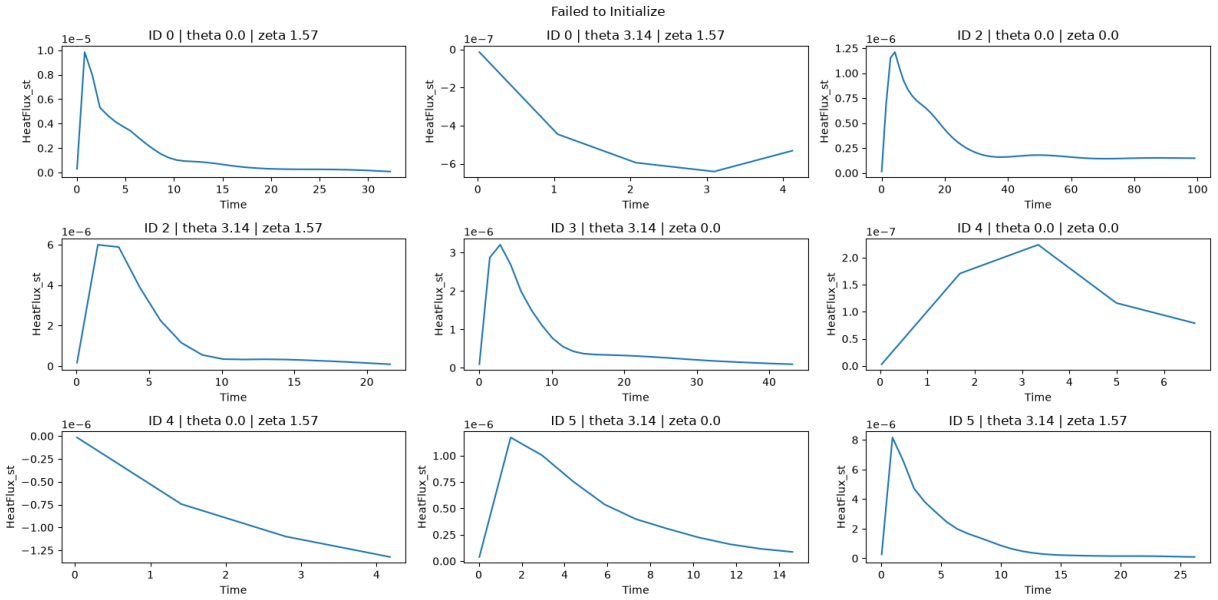
for i, ax in enumerate(axes.flatten()):
    df_i = pugs.load_file(df_failed_init.iloc[i]["file"], variable)
    t = df_i["time"].to_numpy()
    x = df_i[variable].to_numpy()

    ax.plot(t, x)
```

```
    ax.set_title(f"ID {df_failed_init.iloc[i]['id']} | theta {df_failed_init
    ax.set_xlabel("Time")
    ax.set_ylabel(variable)

fig.suptitle(f"Failed to Initialize")

plt.tight_layout()
plt.show()
```



In [8]:
```
fig, axes = plt.subplots(3, 3, figsize=(16,8))

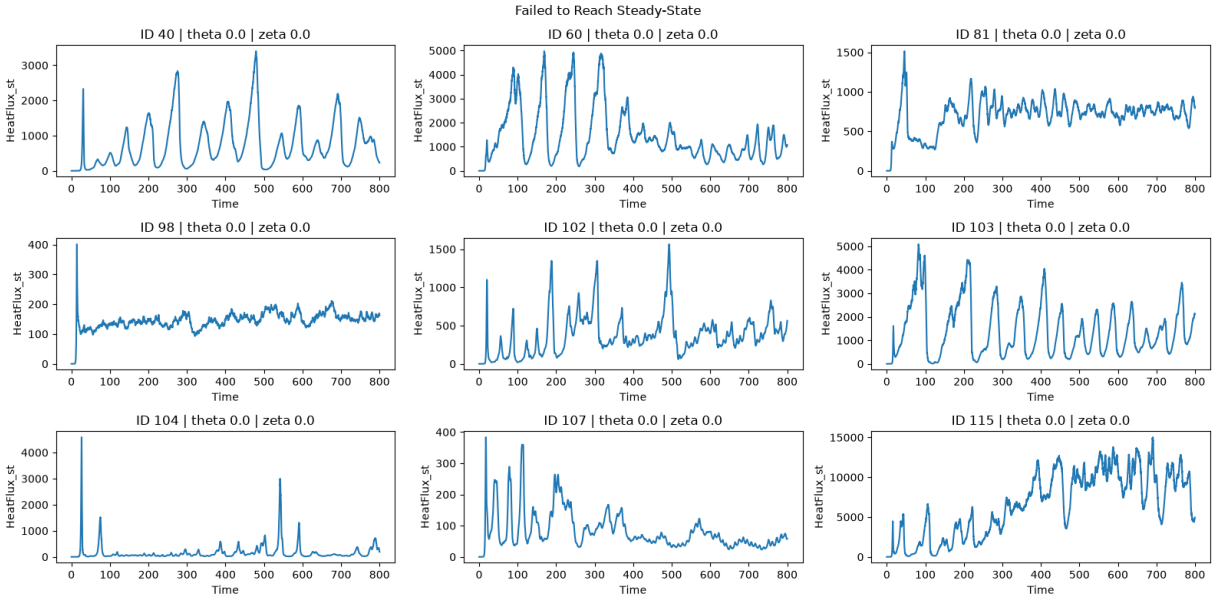
for i, ax in enumerate(axes.flatten()):
    df_i = pugs.load_file(df_failed_algo.iloc[i]["file"], variable)
    t = df_i["time"].to_numpy()
    x = df_i[variable].to_numpy()

    ax.plot(t, x)

    ax.set_title(f"ID {df_failed_algo.iloc[i]['id']} | theta {df_failed_algo
    ax.set_xlabel("Time")
    ax.set_ylabel(variable)

fig.suptitle(f"Failed to Reach Steady-State")

plt.tight_layout()
plt.show()
```



In [9]:
```
logx = np.log(df["start_time"])
logy = np.log(df["mean"])

slope, intercept, r_val, p_val, std_err = st.linregress(logx, logy)

print(f"Mean {variable} = {np.exp(intercept):.5f} * (Time to Steady-State)^(
print(f"R-squared: {r_val**2:.5f}")
```

```
Mean HeatFlux_st = 2536.42827 * (Time to Steady-State)^(-1.47043)
R-squared: 0.68083
```

In [10]:
```
xline = np.linspace(df["start_time"].min(), df["start_time"].max(), 400)
yline = np.exp(intercept) * xline**slope

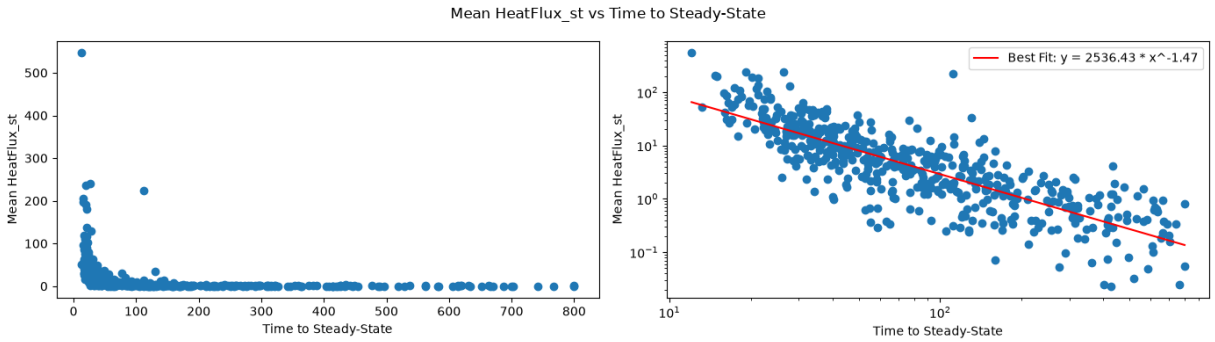
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

for i, ax in enumerate(axes.flatten()):
    ax.scatter(df["start_time"], df["mean"])
    ax.set_xlabel("Time to Steady-State")
    ax.set_ylabel(f"Mean {variable}")

    if i == 1:
        ax.plot(xline, yline, color='red', label=f"Best Fit: y = {np.exp(int
        ax.set_xscale("log")
        ax.set_yscale("log")

fig.suptitle(f"Mean {variable} vs Time to Steady-State")

plt.legend()
plt.tight_layout()
plt.show()
```



In [11]:
```
df = df.dropna(subset=["std", "mean"])
slope, intercept, r_val, p_val, std_err = st.linregress(df["std"], df["mean"

print(f"Mean {variable} = {slope:.5f} * (Standard Deviation) + {intercept:.5
print(f"R-squared: {r_val**2:.5f}")
```

```
Mean HeatFlux_st = 2.66973 * (Standard Deviation) + 4.48841
R-squared: 0.90875
```

In [12]:
```
xline = np.linspace(df["std"].min(), df["std"].max(), 400)
yline = slope * xline + intercept

fig, axes = plt.subplots(1, 2, figsize=(14, 4))

for i, ax in enumerate(axes.flatten()):
    ax.scatter(df["std"], df["mean"])
    ax.set_xlabel("Standard Deviation")
    ax.set_ylabel(f"Mean {variable}")
    ax.plot(xline, yline, color='red', label=f"Best Fit: y = {slope:.2f} * x

    if i == 1:
        ax.set_xlim(-1, 20)
        ax.set_ylim(-5, 100)

fig.suptitle(f"Mean {variable} vs Standard Deviation")

plt.legend()
plt.tight_layout()
plt.show()
```

